

Day 77: Weekly Revision — NumPy, Pandas and Data Cleaning

1. Day Number

Day 77

2. Topic Name

Revision of Days 71–76: NumPy, Pandas, Data Cleaning, Encoding, and Scaling

3. Connection

During Days **71–76**, you moved from basic numerical arrays to working with small datasets.

You learned how to:

NumPy arrays → **array operations** → **Pandas DataFrames** → **inspect data** → **clean data** → **prepare data for ML**

Today, you will combine those ideas in one very small revision exercise.

4. Revision Summary of Days 71–76

Day 71 — NumPy Fundamentals

You learned that a **NumPy array** stores multiple values efficiently.

Example idea:

```
prices = np.array([100, 120, 150, 200])
```

Important ideas:

- NumPy
 - arrays
 - shape
 - dtype
 - one-dimensional arrays
-

Day 72 — NumPy Indexing and Arithmetic

You learned how to access and calculate with array values.

For example:

```
sales[0]  
sales[1:3]  
  
sales.sum()  
sales.mean()
```

You also learned that operations such as:

```
sales + 10
```

can affect every element in the array.

Day 73 — Pandas DataFrames

You learned that a **DataFrame** represents tabular data with rows and named columns.

Example:

name	city	age
Asha	Bengaluru	24
Ravi	Mysuru	30

You also learned how to select a column conceptually:

```
df["city"]
```

Day 74 — Loading and Inspecting Data

Before changing data, you should first understand what it contains.

Useful inspection tools included:

```
df.head()  
df.shape  
df.columns  
df.dtypes
```

Inspection helps you notice unexpected columns, incorrect data types, missing data, or other problems.

Day 75 — Basic Data Cleaning

You learned about common data-quality problems such as:

- missing values
- duplicate rows
- incorrect data types

Useful Pandas tools included:

```
isna()  
fillna()
```

```
dropna()
drop_duplicates()
```

The important lesson was that **cleaning decisions depend on what the data means**.

Day 76 — Basic ML Preprocessing

You learned that machine-learning models often need data to be transformed.

For example:

```
City
Bengaluru
Mysuru
Bengaluru
```

may need to become something numeric, such as:

```
City_Code
0
1
0
```

You also learned the basic ideas behind **scaling** and **normalization**, which help put numerical features into more suitable ranges.

5. Important Topics

For today's revision, remember these seven ideas:

NumPy arrays

Useful for storing and calculating with numerical values.

Pandas DataFrames

Useful for working with data arranged into rows and columns.

Missing values

A value may be unavailable or blank.

Example:

```
age
25
NaN
31
```

Duplicates

The same row might accidentally appear more than once.

Data types

Values should have appropriate types.

For example:

"25"

is text, while:

25

is an integer.

Categorical encoding

Categories such as:

Bengaluru
Mysuru
Hubballi

can be converted into numeric representations when needed for machine learning.

Scaling

Scaling changes the size or range of numerical values while preserving their meaning for later analysis or ML processing.

6. Foundational Notes

A typical beginner data workflow looks like this:

```
Get data
  ↓
Create or load DataFrame
  ↓
Inspect data
  ↓
Find problems
  ↓
Clean data
  ↓
Transform necessary columns
  ↓
Prepare for later analysis or ML
```

A useful rule is:

| Inspect before cleaning, and clean before preprocessing.

For example, suppose you encode a city column before checking the data.

You might accidentally have:

Bengaluru
bengaluru
Bengaluru

Those could incorrectly become different categories.

Cleaning first reduces these problems.

Also remember that **NumPy and Pandas are related but serve slightly different purposes**.

NumPy focuses heavily on arrays and numerical computation.

Pandas builds convenient labeled tables and uses NumPy-style numerical ideas underneath.

7. Easy Example

Imagine this tiny customer dataset:

name	age	city
Asha	25	Bengaluru
Ravi	missing	Mysuru
Ravi	missing	Mysuru
Meera	31	Bengaluru

There are two obvious problems.

First, Ravi's age is missing.

Second, Ravi's row appears twice.

After simple cleaning, the dataset might conceptually become:

name	age	city
Asha	25	Bengaluru
Ravi	28	Mysuru
Meera	31	Bengaluru

Then the city category could be prepared for later ML use:

name	age	city	city_code
Asha	25	Bengaluru	0

Ravi	28	Mysuru	1
Meera	31	Bengaluru	0

The exact number assigned to each city is not important by itself. The goal here is simply to practice converting categories into a machine-readable representation.

8. Revision Problem Statement

Create or load a **tiny customer DataFrame** containing approximately 4–6 rows.

Include at least these columns:

```
customer_name
age
city
```

Your dataset should contain:

- one missing value
- one duplicate row
- one categorical city column

Your program should:

1. Display the original DataFrame.
2. Inspect its first rows.
3. Check its dimensions and data types.
4. Identify the missing value.
5. Fix the missing value using a simple reasonable value.
6. Remove the duplicate row.
7. Convert or prepare the city column into a simple numeric form for later machine-learning use.
8. Display the cleaned/prepared DataFrame.

Keep the dataset very small.

9. Concepts Used

You will practice:

```
NumPy/Pandas data concepts
DataFrame creation
Rows and columns
head()
shape
dtypes
Missing-value detection
```

```
fillna() or another simple cleaning choice
drop_duplicates()
Categorical data
Simple encoding
Basic preprocessing workflow
```

You do **not** need advanced machine-learning libraries for this revision exercise.

10. Thought Process

Before writing code, think through the problem in this order.

Step 1: What does my raw data contain?

Identify the columns.

For example:

```
name
age
city
```

Ask yourself which columns are numerical and which are categorical.

age is numerical.

city is categorical.

Step 2: Is the dataset structured correctly?

Inspect:

```
head()
shape
columns
dtypes
```

Do not immediately start modifying the DataFrame.

Step 3: Are any values missing?

Check the DataFrame for missing values.

If one age is missing, decide on a very simple replacement for this exercise.

Step 4: Are there duplicate rows?

Check whether a customer row appears twice.

If so, remove the duplicate.

Step 5: Is a categorical feature ready for ML?

The city names are text.

A basic ML preprocessing step could create a new numeric column representing those categories.

For example:

Bengaluru	→ 0
Mysuru	→ 1

Step 6: Check the result again

After cleaning, inspect your DataFrame once more.

This lets you confirm that the missing value and duplicate were actually handled.

11. Beginner-Friendly Pseudocode

```
START

import Pandas

create a small customer DataFrame

print original DataFrame

inspect first few rows
inspect shape
inspect column names
inspect data types

check for missing values

replace the missing age
with one simple reasonable value

remove duplicate rows

look at unique city categories

create a simple numeric representation
for each city

store that representation
in a new column

print cleaned and prepared DataFrame
```

END

Notice that this pseudocode describes the **steps**, not the complete Python implementation.

12. Suggested Solving Approach — Pandas-Based Approach

Use Pandas as the main tool for this exercise.

A sensible workflow is:

```
Create DataFrame
    ↓
Inspect
    ↓
Detect missing data
    ↓
Fix missing data
    ↓
Remove duplicates
    ↓
Encode city
    ↓
Inspect final result
```

You can use NumPy ideas where useful, but you do not need to force NumPy into the program.

The main purpose of today's revision is understanding the **data-processing sequence**.

13. Easy Edge Cases

Empty DataFrame

Suppose:

```
df
```

contains no rows.

Then there is nothing useful to clean or encode.

Before performing many operations, you could conceptually check:

```
Is the DataFrame empty?
```

If yes, your program should avoid assuming customer rows exist.

No Missing Values

Your missing-value check might report that every column is complete.

That is fine.

A cleaning program should not assume something is missing.

Conceptually:

```
IF missing values exist
    handle them
ELSE
    continue
```

No Duplicate Rows

`drop_duplicates()` should not cause a problem just because duplicates do not exist.

The DataFrame may simply stay the same.

Only One City

Suppose every customer lives in Bengaluru.

Then the city column has only one category.

A simple encoder may give every row the same encoded value.

14. Common Mistakes to Avoid

Mistake 1: Cleaning before inspecting

Avoid immediately modifying the dataset.

First understand what is wrong.

Mistake 2: Forgetting that `NaN` represents missing data

Do not expect:

```
value == ""
```

to find every kind of missing value.

Pandas provides tools designed specifically for missing data.

Mistake 3: Removing duplicates without understanding them

Two rows having the same city does **not** mean they are duplicates.

For example:

Asha	25	Bengaluru
Meera	30	Bengaluru

These are different customers.

Mistake 4: Encoding categories inconsistently

Do not accidentally use:

Bengaluru → 0

in one place and:

Bengaluru → 2

somewhere else in the same dataset.

Mistake 5: Treating category codes like meaningful quantities

If:

Bengaluru → 0
Mysuru → 1
Hubballi → 2

that does **not** automatically mean:

Hubballi > Mysuru > Bengaluru

The numbers are simply representations of categories in this beginner exercise.

Mistake 6: Forgetting to inspect after cleaning

Always check what your operations produced.

A useful habit is:

inspect → change → inspect again

15. Quick Self-Check Questions

1. What is the main difference between a NumPy array and a Pandas DataFrame?

Think about simple numerical arrays versus labeled rows and columns.

2. Why should you inspect a DataFrame before cleaning it?

Think about knowing what problems actually exist before changing data.

3. Which Pandas concept can help detect missing values?

Recall Day 75.

4. Why might a text column such as city need to be encoded before machine learning?

Think about what kind of input many mathematical models work with.

5. Does assigning `Bengaluru = 0` and `Mysuru = 1` mean **Mysuru** is mathematically greater than **Bengaluru**?

Think about the difference between a **category label** and a true numerical quantity.

16. Hint Only

Start with something similar to this structure:

```
import pandas as pd

data = {
    "customer_name": [...],
    "age": [...],
    "city": [...]
}

df = pd.DataFrame(data)
```

Then work through the problem in this order:

```
head()
↓
shape / dtypes
↓
find missing value
↓
fill missing value
↓
remove duplicate
↓
create city-to-number mapping
↓
create encoded city column
↓
print final DataFrame
```

For the city column, think about a small mapping such as:

```
city_mapping = {  
    "Bengaluru": ...,  
    "Mysuru": ...  
}
```

Your challenge is to decide **how to apply that mapping to the DataFrame**.

Do not add scaling unless you want extra revision practice. The core Day 77 exercise is to inspect, clean, remove the duplicate, and prepare the categorical feature without building a full ML model.